

Q1. Backpropagation Weight Update

Given:

Input $x = 1$, target $t = 0.8$, output $o = 0.6$, learning rate $\eta = 0.5$

Concept:

Backpropagation updates weight based on error:

$$\Delta w = \eta(t - o)x$$

Step 1: Compute Error

$$t - o = 0.8 - 0.6 = 0.2$$

Step 2: Multiply with input

$$0.2 \times 1 = 0.2$$

Step 3: Apply learning rate

$$\Delta w = 0.5 \times 0.2 = 0.1$$

✅ **Final Answer:** $\Delta w = 0.1$

📌 **Learning Tip:**

If output is less than target → weight increases.

◆ Q2. Updated Weight

Given:

Initial weight $w = 0.4$, update $\Delta w = 0.1$

Concept:

$$w_{new} = w + \Delta w$$

Step:

$$w_{new} = 0.4 + 0.1 = 0.5$$

✅ **Final Answer:** 0.5

 Learning Tip:

Positive update → weight increases.

◆ **Q3. Momentum Method**

Given:

$\eta = 0.3$, $\alpha = 0.9$, gradient = 0.2, previous update = 0.1

Concept:

Momentum accelerates learning:

$$\Delta w(t) = \eta \cdot \text{gradient} + \alpha \cdot \Delta w(t - 1)$$

Step 1: Gradient term

$$0.3 \times 0.2 = 0.06$$

Step 2: Momentum term

$$0.9 \times 0.1 = 0.09$$

Step 3: Add both

$$0.06 + 0.09 = 0.15$$

✓ **Final Answer:** 0.15

 Learning Tip:

Momentum helps avoid slow convergence.

◆ **Q4. Mean Squared Error**

Given:

$t = 1$, $o = 0.7$

Concept:

$$E = \frac{1}{2}(t - o)^2$$

Step 1: Error difference

$$1 - 0.7 = 0.3$$

Step 2: Square

$$(0.3)^2 = 0.09$$

Step 3: Multiply by 1/2

$$E = 0.5 \times 0.09 = 0.045$$

✅ **Final Answer:** 0.045

📌 **Learning Tip:**

Square ensures error is always positive.

◆ **Q5. Gradient (Delta)**

Given:

$t = 0.9$, $o = 0.5$, derivative = 0.25

Concept:

$$\delta = (t - o) \cdot f'(o)$$

Step 1: Error

$$0.9 - 0.5 = 0.4$$

Step 2: Multiply derivative

$$0.4 \times 0.25 = 0.1$$

✅ **Final Answer:** $\delta = 0.1$

📌 **Learning Tip:**

Delta shows how much output neuron contributed to error.

◆ **Q6. Weight Update using Gradient**

Given:

$\eta = 0.2$, $\delta = 0.1$, $x = 2$

Concept:

$$\Delta w = \eta \cdot \delta \cdot x$$

Step:

$$0.2 \times 0.1 \times 2 = 0.04$$

✅ **Final Answer:** 0.04

📌 **Learning Tip:**

Higher input $\rightarrow$ larger weight change.

◆ Q7. Net Input

Given:

$$x_1 = 1, x_2 = 2, w_1 = 0.5, w_2 = 0.3, \text{ bias } b = 0.2$$

Concept:

$$net = \sum xw + b$$

Step:

$$(1)(0.5) + (2)(0.3) + 0.2 = 0.5 + 0.6 + 0.2 = 1.3$$

✅ **Final Answer:** 1.3

📌 **Learning Tip:**

Net input is weighted sum + bias.

◆ Q8. Sigmoid Output

Given:

$$net = 1.3$$

Concept:

$$o = \frac{1}{1 + e^{-x}}$$

Step:

$$o = \frac{1}{1 + e^{-1.3}} \approx 0.785$$

✅ **Final Answer:** 0.785

📌 **Learning Tip:**

Sigmoid output always lies between 0 and 1.

◆ Q9. Output Layer Delta

Given:

$$t = 1, o = 0.785$$

Concept:

$$\delta = (t - o) \cdot o(1 - o)$$

Step 1: Derivative

$$0.785(1 - 0.785) = 0.168$$

Step 2: Error

$$1 - 0.785 = 0.215$$

Step 3: Multiply

$$0.215 \times 0.168 \approx 0.036$$

✅ **Final Answer:** $\delta \approx 0.036$

📌 **Learning Tip:**

Sigmoid derivative simplifies computation.

◆ Q10. Bias Update

Given:

$$\eta = 0.5, \delta = 0.036$$

Concept:

$$\Delta b = \eta \cdot \delta$$

Step:

$$0.5 \times 0.036 = 0.018$$

✅ **Final Answer:** 0.018

📌 **Learning Tip:**

Bias update does not depend on input.

Q11. Hebbian Weight Update

Given:

Input $x = [1, -1]$, output $y = 1$, learning rate $\eta = 0.5$

Concept:

Hebbian rule:

$$\Delta w = \eta \cdot x \cdot y$$

👉 “Neurons that fire together, wire together”

Step 1: Multiply input with output

$$x \cdot y = [1, -1] \times 1 = [1, -1]$$

Step 2: Multiply with learning rate

$$\Delta w = 0.5 \times [1, -1] = [0.5, -0.5]$$

✅ **Final Answer:** $[0.5, -0.5]$

📌 **Learning Tip:**

Positive correlation → weight increases, negative → decreases.

♦ Q12. Hebbian Learning Over Iterations

Given:

Initial weight = 0, $x = 1$, $y = 1$, $\eta = 0.5$, iterations = 3

Concept:

Weight keeps increasing with each iteration.

Step 1: Single update

$$\Delta w = 0.5 \times 1 \times 1 = 0.5$$

Step 2: After 3 iterations

$$w = 0 + 3 \times 0.5 = 1.5$$

✅ **Final Answer:** 1.5

📌 **Learning Tip:**

Hebbian learning can cause weights to grow indefinitely.

◆ Q13. Associative Memory Weight Matrix

Given:

Pattern $X = [1, -1]$

Concept:

Weight matrix:

$$W = X^T X$$

Step 1: Convert to column vector

$$X^T = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 2: Multiply

$$W = \begin{bmatrix} 1 \\ -1 \end{bmatrix} [1, -1] = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Step 3 (Hopfield rule):

Diagonal = 0

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

✅ **Final Answer:** Matrix above

📌 **Learning Tip:**

This matrix stores the pattern in memory.

◆ **Q14. Associative Recall**

Given:

$$W = [1, -1], X = [1, 1]$$

Concept:

Output = dot product

Step:

$$Output = (1)(1) + (1)(-1) = 1 - 1 = 0$$

✓ **Final Answer: 0**

📌 **Learning Tip:**

If output is close to stored pattern → successful recall.

◆ **Q15. Hopfield Energy**

Given:

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}, S = [1, -1]$$

Concept:

$$E = -\frac{1}{2} \sum w_{ij} s_i s_j$$

Step 1: Compute terms

$$(-1)(1)(-1) = 1$$

$$(-1)(-1)(1) = 1$$

Step 2: Sum

$$1 + 1 = 2$$

Step 3: Apply formula

$$E = -\frac{1}{2} \times 2 = -1$$

✓ **Final Answer:** -1

✦ **Learning Tip:**

Lower energy $\rightarrow$ more stable state.

◆ **Q16. Energy Change**

Given:

Energy: $-2 \rightarrow -5$

Concept:

Hopfield network minimizes energy.

Step:

$$\Delta E = -5 - (-2) = -3$$

Since energy decreases $\rightarrow$ stability improves

✓ **Final Answer:** Moving toward stability

✦ **Learning Tip:**

Energy always decreases during convergence.

◆ **Q17. Stability Condition**

Given:

No change in neuron states

Concept:

Stable state = minimum energy

Explanation:

If output does not change after update $\rightarrow$ system reached equilibrium

✓ **Final Answer:** Stable equilibrium

✦ **Learning Tip:**

This is final stored memory state.

◆ **Q18. Euclidean Distance**

Given:

$$X = [2,3], W = [1,1]$$

Concept:

$$D = \sqrt{(x_1 - w_1)^2 + (x_2 - w_2)^2}$$

Step:

$$D = \sqrt{(2 - 1)^2 + (3 - 1)^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.23$$

✓ **Final Answer:** ≈ 2.23

✦ **Learning Tip:**

Smaller distance = closer neuron.

◆ Q19. Best Matching Unit (BMU)

Given:

$$X = [3,4], W_1 = [1,1], W_2 = [3,3]$$

Step 1: Distance to W_1

$$\sqrt{(3 - 1)^2 + (4 - 1)^2} = \sqrt{13}$$

Step 2: Distance to W_2

$$\sqrt{(3 - 3)^2 + (4 - 3)^2} = \sqrt{1} = 1$$

Step 3: Compare

$$1 < \sqrt{13}$$

✓ **Final Answer:** BMU = W_2

✦ **Learning Tip:**

BMU is neuron closest to input.

◆ Q20. SOM Weight Update

Given:

$$W = [0.5, 0.5], X = [1, 1], \eta = 0.2$$

Concept:

$$W_{new} = W + \eta(X - W)$$

Step 1: Difference

$$[1,1] - [0.5,0.5] = [0.5,0.5]$$

Step 2: Multiply

$$0.2 \times [0.5,0.5] = [0.1,0.1]$$

Step 3: Add

$$[0.5,0.5] + [0.1,0.1] = [0.6,0.6]$$

✅ **Final Answer:** [0.6, 0.6]

📌 **Learning Tip:**

Weights move toward input.

◆ **Q21. LVQ (Correct Classification)**

Given:

$$W = [2,2], X = [3,3], \eta = 0.1$$

Concept:

If classification is correct, weights move **toward input**:

$$W_{new} = W + \eta(X - W)$$

Step 1: Difference

$$X - W = [3,3] - [2,2] = [1,1]$$

Step 2: Multiply

$$0.1 \times [1,1] = [0.1,0.1]$$

Step 3: Update

$$[2,2] + [0.1,0.1] = [2.1,2.1]$$

✓ **Final Answer:** [2.1, 2.1]

📌 **Learning Tip:**

Correct classification → move closer to input.

◆ **Q22. LVQ (Incorrect Classification)**

Concept:

If classification is wrong, weights move **away from input**:

$$W_{new} = W - \eta(X - W)$$

Step 1: Difference

$$[1, 1]$$

Step 2: Multiply

$$0.1 \times [1, 1] = [0.1, 0.1]$$

Step 3: Update

$$[2, 2] - [0.1, 0.1] = [1.9, 1.9]$$

✓ **Final Answer:** [1.9, 1.9]

📌 **Learning Tip:**

Wrong classification → push weights away.

◆ **Q23. K-Fold Cross Validation**

Given:

Total samples = 100, $K = 5$

Concept:

Dataset divided into equal parts.

Step 1: Fold size

$$100/5 = 20$$

Step 2: Training samples

$$100 - 20 = 80$$

✅ Final Answer:

Fold size = 20, Training = 80

📌 Learning Tip:

Each fold is used once as test set.

◆ Q24. Generalization Check

Given:

Training error = 5%, Testing error = 20%

Concept:

Compare training vs testing error.

Analysis:

Difference is moderate (not extreme)

👉 Model performs reasonably well on unseen data

✅ Final Answer: Good generalization

📌 Learning Tip:

Small gap = good model.

◆ Q25. Overfitting Identification

Given:

Training accuracy = 98%, Testing = 60%

Concept:

Large gap between training and testing.

Analysis:

Model memorized training data but fails on new data

✅ Final Answer: Overfitting

📌 Learning Tip:

Very high training + low testing = danger sign.

◆ **Q26. Network Pruning**

Given:

Total weights = 100, removed = 30

Step 1: Remaining

$$100 - 30 = 70$$

Step 2: Percentage reduction

$$(30/100) \times 100 = 30\%$$

✓ **Final Answer:**

Remaining = 70, Reduction = 30%

📌 **Learning Tip:**

Pruning reduces complexity and overfitting.

◆ **Q27. Learning Rate Decay**

Given:

Initial $\eta = 0.5$, decay = 0.9

Concept:

$$\eta_{new} = \eta \times decay$$

Step:

$$0.5 \times 0.9 = 0.45$$

✓ **Final Answer:** 0.45

📌 **Learning Tip:**

Learning rate decreases over time for stability.

◆ **Q28. Hessian Matrix**

Given:

$$E = x^2 + y^2$$

Concept:

Hessian = matrix of second derivatives

Step 1: Derivatives

$$\frac{\partial^2 E}{\partial x^2} = 2, \frac{\partial^2 E}{\partial y^2} = 2$$

Cross derivatives = 0

Step 2: Matrix

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

✓ **Final Answer:** Matrix above

📌 **Learning Tip:**

Hessian shows curvature of error surface.

◆ **Q29. BAM Weight Matrix**

Given:

$$X = [1, -1], Y = [1, 1]$$

Concept:

$$W = X^T Y$$

Step 1: Convert

$$X^T = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 2: Multiply

$$W = \begin{bmatrix} 1 \\ -1 \end{bmatrix} [1, 1] = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

✓ **Final Answer:** Matrix above

📌 **Learning Tip:**

BAM stores input-output associations.

◆ Q30. Stable Convergence

Given:

Output does not change after updates

Concept:

Network reached minimum energy state

Explanation:

No further updates needed → learning complete

✅ **Final Answer:** Stable equilibrium / convergence

📌 **Learning Tip:**

This is final memory state in networks.